# מדריך Git ו-GitHub לתלמידים

Oct 19, 2026 · @Alona Klaiman

## 1. מה זה Git ומה זה GitHub?

הוא אתר באינטרנט GitHub. הוא תוכנה ששומרת את כל ההיסטוריה של הפרויקט שלכם במחשב Git שבו שומרים עותק של הפרויקט הזה, משתפים אותו ועובדים עליו יחד עם אחרים.

### Git – מכונת זמן" לקבצים"

דמיינו שאתם כותבים עבודה גדולה ושומרים קבצים בשמות כמו `עבודה_סופית.docx`, אחרי שבוע כבר אי אפשר לדעת מה `עבודה_סופית_2_מתוקן.docx`, `עבודה_סופית_באמת.docx`. השתנה ומתי.

זוכר כל "נקודת שמירה" שיצרתם: מה Git-פותר את זה. במקום עשרות עותקים, יש תיקייה אחת, ו Git השתנה, מי שינה, מתי ולמה. אפשר לחזור לכל נקודה בעבר, כמו נקודות שמירה במשחק מחשב.

- Git נוצר ב-2005 על ידי לינוס טורבאלדס, מי שיצר גם את מערכת ההפעלה Linux.
- Git עובד על המחשב שלכם, גם בלי אינטרנט.
- Git הוא חינמי וקוד פתוח.

### Hub – "המרכז"

הוא GitHub. באנגלית פירושה מרכז או צומת, כמו תחנה מרכזית שכל הקווים נפגשים בה Hub המילה "המרכז של Git": אתר שבו מאחסנים פרויקטים שמנוהלים ב-Git.

אנלוגיה פשוטה:

| | Git | GitHub |
|---|---|---|
| מה זה | תוכנה | אתר אינטרנט (שירות ענן) |
| איפה זה רץ | על המחשב שלכם | על השרתים של GitHub |
| דומה ל... | המצלמה שמצלמת כל שלב בעבודה | האלבום המשותף ב"ענן" שכולם רואים |
| צריך אינטרנט? | לא | כן |
| מי מפעיל | אתם, בפקודות או דרך VS Code | חברת GitHub (בבעלות Microsoft) |

### למה זה חשוב לכם?

- **גיבוי:** המחשב התקלקל? הפרויקט שמור ב-GitHub.
- **עבודת צוות:** כמה תלמידים עובדים על אותו פרויקט בלי לדרוס אחד לשני את הקבצים.
- **תיק עבודות:** חשבון GitHub עם פרויקטים הוא "קורות חיים" של מתכנתים. מעסיקים, תוכניות מצוינות ויחידות טכנולוגיות בצבא מסתכלים עליו.
- **סטנדרט בתעשייה:** כמעט כל חברת הייטק משתמשת ב-Git.

## 2. מושגי יסוד – המילון של Git

כדאי להכיר אותן לפני שמתחילים להקליד ב-Git. שבע מילים מספיקות כדי להבין 90% ממה שקורה בפקודות.

| מושג | בעברית | הסבר | דוגמה מחיי בית הספר |
|---|---|---|---|
| Repository (Repo) | מאגר | תיקיית הפרויקט + כל ההיסטוריה שלה | התיקייה של עבודת הגמר בפיזיקה |
| Commit | נקודת שמירה | "צילום" של מצב הקבצים ברגע מסוים, עם הודעה שמסבירה מה השתנה | "הוספתי את פרק המבוא" |
| Staging (add) | הכנה לשמירה | בחירה אילו שינויים ייכנסו לצילום הבא | לבחור אילו דפים להכניס למעטפה לפני ששולחים |
| Push | דחיפה | שליחת נקודות השמירה מהמחשב ל-GitHub | להעלות את העבודה לענן |
| Pull | משיכה | הורדת שינויים חדשים מ-GitHub למחשב | לקבל את מה שחבר לצוות הוסיף |
| Clone | שכפול | הורדה של מאגר שלם מ-GitHub למחשב בפעם הראשונה | להוריד את הפרויקט למחשב חדש |
| Branch | ענף | קו פיתוח נפרד, כדי לנסות משהו בלי לקלקל את הגרסה הראשית | לנסות עיצוב חדש לאתר בלי לגעת באתר שעובד |
| Merge | מיזוג | איחוד ענף בחזרה לגרסה הראשית | העיצוב החדש עבד, מצרפים אותו |

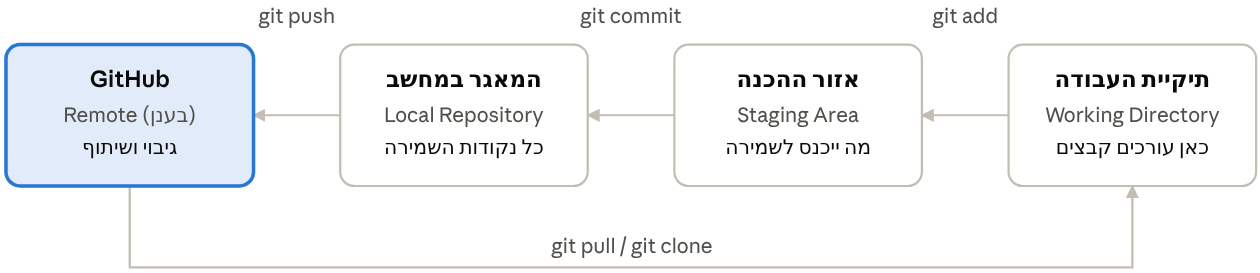
### מחזור העבודה הבסיסי

כל עבודה ב-Git חוזרת על אותם ארבעה צעדים:

1. **עורכים** קבצים בתיקייה (Working Directory).
2. **בוחרים** מה לשמור: `git add`.
3. **שומרים** נקודת שמירה עם הסבר: `git commit -m "הסבר"`.
4. **מעלים** ל-GitHub: `git push`.

השרטוט מראה את המסלול הזה:

**כל שינוי עובר שלושה צעדים עד שהוא נשמר בענן**




אזורים, 4 פקודות 4 · Git-מחזור העבודה ב

מחזירים אותם `git clone`-ו `git pull`; השינויים זזים מימין לשמאל עד שהם מגיעים ל-GitHub; מהענף אל המחשב.

## 3. התקנת Git

פעם אחת במחשב, מגדירים שם ואימייל, וזהו. ההתקנה לוקחת כ-5 דקות Git מתקינים את.

### ב-Windows

1. נכנסים לאתר הרשמי git-scm.com ולוחצים על **Download for Windows**.
2. מריצים את קובץ ההתקנה שירד (למשל `Git-2.xx.x-64-bit.exe`).
3. לוחצים **Next** בכל המסכים. ברירות המחדל מתאימות למתחילים.
   - במסך בחירת העורך אפשר לבחור **Use Visual Studio Code as Git's default editor** אם VS Code כבר מותקן.
4. בסיום, פותחים את התוכנה **Git Bash** מתפריט ההתחלה. זה חלון שחור שבו מקלידים פקודות (טרמינל).

### ב-Mac

1. פותחים את אפליקציית **Terminal** (חיפוש עם `Cmd + רווח` והקלדת Terminal).
2. מקלידים:

```
git --version
```

3. אם Git לא מותקן, ה-Mac יציע להתקין את "Command Line Developer Tools". מאשרים ומחכים לסיום.
4. אפשרות נוספת: להוריד מתקין מ-git-scm.com.

### בדיקה שההתקנה הצליחה

מקלידים (Git Bash ב-Windows, Terminal ב-Mac) בטרמינל:

```
git --version
```

אם מופיעה שורה כמו `git version 2.xx.x` – הכול עובד.

### הגדרה ראשונית (חובה, פעם אחת)

צריך לדעת מי אתם, כדי לחתום את השם שלכם על כל נקודת שמירה. מקלידים את שתי השורות, עם השם והאימייל שלכם:

```
git config --global user.name "Noa Levi"
git config --global user.email "noa.levi@example.com"
```

חשוב: השתמשו באותו אימייל שתפתחו איתו את חשבון GitHub. כך GitHub יזהה שהשמירות שלכם.

עוד הגדרה מומלצת, כדי שהענף הראשי ייקרא `main` כמו ב-GitHub:

```
git config --global init.defaultBranch main
```

בדיקה של ההגדרות:

```
git config --list
```

## 4. פתיחת חשבון GitHub ויצירת מאגר ראשון

חשבון GitHub הוא חינמי, ומותר לפתוח אותו מגיל 13 (תנאי השימוש של GitHub). תלמידי י'–י"א עומדים בתנאי הזה.

### פתיחת החשבון

1. נכנסים ל-github.com ולוחצים **Sign up**.
2. בפרק הקודם Git-מזינים אימייל – אותו אימייל שהגדרתם ב.
3. בוחרים סיסמה חזקה (לפחות 15 תווים, או 8 תווים עם מספר ואות קטנה).
4. הוא יופיע בכתובת של כל הפרויקטים שלכם, למשל **שם משתמש (Username)** בוחרים `github.com/noa-levi`.
   - טיפ: בחרו שם רציני ומקצועי. בעוד שנתיים תשלחו את הקישור הזה למעסיקים או לצבא.
5. פותרים את חידת האימות, ומאשרים את הקוד שנשלח לאימייל.
6. **אימות דו-שלבי** מומלץ מאוד להפעיל **(2FA)** בהגדרות: Settings → Password and authentication.

### הטבה לתלמידים: GitHub Education

תלמידים מגיל 13 ומעלה, שלומדים בבית ספר תיכון או במוסד לימודים אחר, יכולים לבקש את **Student Developer Pack** – חבילת כלים חינמית למפתחים (פרטי התוכנית). צריך כתובת אימייל של בית הספר – או צילום של תעודת תלמיד. מגישים ב-education.github.com.

### יצירת מאגר (Repository) ראשון

1. אחרי ההתחברות, לוחצים על **+** בפינה העליונה ← **New repository**.
2. ממלאים:
   - **Repository name:** שם באנגלית, בלי רווחים. למשל `my-first-project`.
   - **Description:** משפט קצר על הפרויקט.
   - **Public / Private:** ציבורי (כולם רואים) או פרטי (רק אתם ומי שתזמינו). לעבודות בית ספר שאסור להעתיק מהן – בחרו Private.
   - סמנו **Add a README file**. זה קובץ שמסביר מה יש בפרויקט, והוא מוצג בעמוד הראשי של המאגר.
3. לוחצים **Create repository**.

זהו – יש לכם מאגר ראשון בענן. הכתובת שלו תהיה בערך כזו: `https://github.com/noa-levi/my-first-project`.

## 5. העלאת קבצים ל-GitHub

יש שתי דרכים להעלות קבצים: גרירה דרך האתר (הכי פשוט, טוב להתחלה) ופקודות מהטרמינל Git (הדרך המקצועית, שומרת היסטוריה מלאה).

### דרך א': העלאה דרך האתר

1. נכנסים לעמוד המאגר ב-GitHub.
2. לוחצים **Add file** ← **Upload files**.
3. גוררים קבצים או תיקיות לחלון, או לוחצים **choose your files**.
4. בתחתית העמוד כותבים הודעת שמירה (Commit message), למשל `העלאת מצגת הפרויקט`.
5. לוחצים **Commit changes**.

מגבלה: דרך הדפדפן אפשר להעלות קובץ של עד 25MB בלבד ([GitHub Docs]).

### דרך ב': העלאה מהטרמינל

זו הדרך שמתכנתים משתמשים בה כל יום. נניח שיש לכם במחשב תיקייה בשם `my-first-project` עם קבצים.

**שלב 1 – הורדת המאגר מ-GitHub למחשב (פעם אחת):**

```
cd Desktop
git clone https://github.com/noa-levi/my-first-project.git
cd my-first-project
```

- `cd` = Change Directory, מעבר לתיקייה.
- `git clone` מוריד את המאגר ויוצר תיקייה עם אותו שם.

**שלב 2 – מוסיפים קבצים לתיקייה** (מעתיקים לתוכה קבצים, או יוצרים חדשים).

**שלב 3 – בודקים מה השתנה:**

```
git status
```

קבצים חדשים יופיעו באדום תחת `Untracked files`.

**שלב 4 – בוחרים מה לשמור:**

```
git add index.html      # קובץ אחד
git add .               # כל הקבצים שהשתנו
```

**שלב 5 – יוצרים נקודת שמירה:**

```
git commit -m "Add home page"
```

**שלב 6 – מעלים ל-GitHub:**

```
git push
```

לא מקבל את הסיסמה הרגילה GitHub. בדפדפן GitHub בפעם הראשונה יקפוץ חלון התחברות של Git). זה מגיע מובנה בהתקנת Windows-ב) בטרמינל; ההתחברות נעשית דרך הדפדפן.

והקבצים שם – GitHub-מרעננים את עמוד המאגר ב.

## פרויקט שכבר קיים במחשב (clone בלי)

אם יש לכם תיקייה קיימת ורוצים לחבר אותה למאגר ריק חדש ב-GitHub (**בלי** שנוצר README):

```
cd my-project
git init
git add .
git commit -m "First commit"
git branch -M main
git remote add origin https://github.com/noa-levi/my-project.git
git push -u origin main
```

- `git init` הופך תיקייה רגילה למאגר Git.
- `git remote add origin` אומר ל-Git שהמאגר נמצא באתר GitHub.
- `-u` זוכר את החיבור, כך שבפעם הבאה מספיק `git push`.

## אילו קבצים אפשר להעלות?

מצטיין בקבצי טקסט, כי הוא מראה בדיוק איזו שורה השתנתה. אבל Git לא מגביל סוגים, רק גודל. כל סוג קובץ שורה (תמונה, Word) הוא רק יודע ש"הקובץ השתנה".

| סוג | דוגמאות | מתאים ל-Git? |
|---|---|---|
| קוד | `.py` `.js` `.html` `.css` `.java` `.cpp` | מצוין – רואים כל שינוי בשורה |
| טקסט ותיעוד | `.md` `.txt` `.json` `.csv` | מצוין |
| מחברות Python | `.ipynb` | טוב |
| תמונות | `.png` `.jpg` `.svg` | בסדר, בכמות סבירה (`.svg` הוא טקסט) |
| מסמכי Office ו-PDF | `.docx` `.pptx` `.pdf` | אפשר, אבל אי אפשר לראות מה השתנה בפנים |

| סוג | דוגמאות | מתאים ל-Git? |
|---|---|---|
| וידאו, קבצי משחק גדולים | `.mp4` `.zip` | לא מומלץ – מגיעים מהר למגבלת הגודל |

### מגבלות גודל

GitHub לפי התיעוד של:

| מגבלה | גודל |
|---|---|
| קובץ דרך הדפדפן | עד 25MB |
| Git מזהיר | מעל 50MB |
| GitHub חוסם קובץ | מעל 100MB |
| קבצים גדולים יותר | רק עם כלי מיוחד בשם Git LFS |

### .gitignore מה אסור להעלות – קובץ

מספרי תעודת זהות או מידע אישי. גם במאגר ,(API keys) לעולם אל תעלו סיסמאות, מפתחות גישה פרטי – זה מסוכן.

ורושמים בו מה `.gitignore` יתעלם מקבצים מסוימים, יוצרים בתיקיית הפרויקט קובץ בשם Git-כדי ש לא להעלות:

```
# קבצי סודות
.env
secrets.txt

# תיקיות שנוצרות אוטומטית
node_modules/
__pycache__/

# קבצי מערכת
.DS_Store
Thumbs.db
```

# 6. חיבור VS Code ל-Git ול-GitHub

VS Code הוא עורך קוד חינמי של Microsoft, ו-Git מובנה בתוכו. אחרי החיבור אפשר לעשות commit ו-push בלחיצת כפתור, בלי להקליד פקודות.

## התקנת VS Code

1. מורידים מ-code.visualstudio.com ומתקינים.
2. ב-Windows, במסך ההתקנה מומלץ לסמן **Add to PATH** ו-**Open with Code**.
3. VS Code מזהה לבד את Git 3 שהותקן בפרק.

## התחברות לחשבון GitHub מתוך VS Code

1. בפינה השמאלית התחתונה (**Accounts**) לוחצים על סמל האדם.
2. בוחרים **Sign in with GitHub** (או "Backup and Sync Settings" ← GitHub).
3. הדפדפן נפתח, מאשרים, וחוזרים ל-VS Code.

## שכפול (Clone) מאגר מ-GitHub דרך VS Code

1. פותחים את לוח הפקודות: `Ctrl+Shift+P` (ב-Mac: `Cmd+Shift+P`).
2. מקלידים `Git: Clone` ובוחרים **Clone from GitHub**.
3. בוחרים את המאגר מהרשימה, ואת התיקייה במחשב שבה הוא יישמר.
4. לוחצים **Open** – הפרויקט נפתח.

## מחזור עבודה ב-VS Code – בלי טרמינל

בסרגל השמאלי יש סמל של ענף (**Source Control**, קיצור: `Ctrl+Shift+G`). שם רואים את כל השינויים.

1. קובץ ששומרים (`Ctrl+S`) **עורכים**. הקובץ יופיע ברשימת Changes עם האות **M** (Modified – שונה) או **U** (Untracked – חדש).
2. ברשימה – נפתחת השוואה: מימין הגרסה החדשה, משמאל הישנה. שורות שנוספו בירוק, שורות שנמחקו באדום.
3. ליד הקובץ **+ לוחצים** = `git add`.
4. בתיבה העליונה ולוחצים **Commit כותבים הודעה** = `git commit`.
5. **Sync Changes לוחצים** (או Push) = `git push`.

| פעולה ב-VS Code | הפקודה המקבילה בטרמינל |
|---|---|
| + ליד קובץ (Stage) | `git add <file>` |
| כפתור Commit | `git commit -m "..."` |
| Sync Changes / Push | `git push` |
| Pull (בתפריט ...) | `git pull` |
| Create new branch ← שם הענף בפינה התחתונה | `git branch` + `git switch` |

### טרמינל VS Code בתוך

פותח טרמינל (Tab המקש שמעל) `Ctrl+\`` :VS Code-אפשר גם להקליד פקודות בלי לצאת מ בתחתית המסך, כבר בתוך תיקיית הפרויקט.

### תוספים מומלצים

לפתיחת חנות התוספים `Ctrl+Shift+X` לוחצים:

- **GitLens** – מראה ליד כל שורה מי כתב אותה ומתי.
- **GitHub Pull Requests** – ניהול בקשות מיזוג (Pull Requests) מתוך VS Code.
- **Claude Code** – עוזר בינה מלאכותית שכותב ועורך קוד. מפורט בפרק הבא.

## 7. Claude Code ב-VS Code – ושמירה ב-GitHub

Claude Code של Anthropic הוא עוזר בינה מלאכותית שעובד בתוך תיקיית הפרויקט: הוא קורא את הקבצים, כותב ועורך קוד, מריץ פקודות, ויודע גם לעשות commit ו-push ל-GitHub לפי בקשה בשפה חופשית.

### לפני שמתחילים – תנאי גיל וחשבון (חשוב למורים)

- **גיל:** לפי תנאי השימוש של Anthropic לחשבונות פרטיים, המשתמש חייב להיות בן 18 לפחות (Consumer Terms). תלמידי י'–י"א אינם יכולים לפתוח חשבון פרטי בעצמם.
- **מנוי:** התוסף דורש חשבון Claude בתשלום (Pro, Max, Team או Enterprise) או חשבון Console (התיעוד הרשמי).
- **המשמעות בכיתה:** את הפרק הזה מומלץ להעביר כ**הדגמה של המורה** על המחשב שלו, מול הכיתה. אם בית הספר או הרשות שוקלים מתן גישה לתלמידים דרך חשבון ארגוני – יש לבדוק מול אילו תנאים חלים על קטינים בהקשר החינוכי. איני יודע אם קיים כיום מסלול ייעודי לבתי Anthropic

ספר תיכוניים.

### התקנת התוסף

לפי התיעוד הרשמי:

1. ב-VS Code לוחצים `Ctrl+Shift+X` (ב-Mac: `Cmd+Shift+X`) לפתיחת התוספים.
2. מחפשים **Claude Code** (של Anthropic) ולוחצים **Install**.
3. פותחים קובץ כלשהו בפרויקט. בפינה הימנית העליונה של העורך מופיע **סמל הניצוץ** (Spark) – בשורת המצב בתחתית החלון. דרך נוספת: לוחצים עליו ✻ **Claude Code**.
4. בפעם הראשונה לוחצים **Sign in** ומאשרים בדפדפן.
5. אם התוסף לא מופיע: `Ctrl+Shift+P` ← **Developer: Reload Window**.

### מותר לו לעשות Claude שלושה מצבי עבודה – מה

בתחתית תיבת הכתיבה יש מחוון מצב. לחיצה עליו מחליפה מצב:

| מצב | מה קורה | מתי להשתמש |
|---|---|---|
| **Plan** | Claude מתאר מה הוא מתכנן ומחכה לאישור לפני שינוי | ללמידה – רואים איך הוא חושב |
| **Manual** | לפני כל עריכת קובץ מוצגת השוואה, ומאשרים או דוחים | מומלץ בכיתה – שליטה מלאה |
| **Auto / Edit automatically** | Claude עורך בלי לשאול | למשתמשים מנוסים |

### דוגמה מלאה: בונים אתר אישי ושומרים ב-GitHub

**שלב 1 – הכנה.** יוצרים ב-GitHub מאגר בשם `my-website` (עם README), ומשכפלים אותו ב-VS Code כמו בפרק 6.

**שלב 2 – בקשה ראשונה.** פותחים את פאנל Claude ומקלידים:

```
צור לי דף אינטרנט אישי בקובץ index.html, עם כותרת, פסקה עליי,
ורשימת תחביבים ותמונה. הוסף קובץ style.css עם עיצוב נקי וצבעים כחולים.
הדף צריך להיות בעברית, מימין לשמאל.
```

תראו את התוכן ותאשרו כל קובץ Manual ייצור את שני הקבצים. במצב Claude.

**שלב 3 – בדיקה.** לוחצים קליק ימני על `index.html` ← **Reveal in File Explorer**, ופותחים בדפדפן.

**שלב 4 – שיפור.** ממשיכים לבקש:

```
הוסף כפתור שמחליף בין מצב בהיר למצב כהה.
```

**שלב 5 – שמירה ב-GitHub, בבקשה בשפה חופשית:**

```
עשה commit ואז, לשינויים עם הודעה שמתארת מה נוסף push ל-GitHub.
```

הוא יבקש אישור לפני כל Manual במצב. `git push`-ו `git commit`, `git add` יריץ בעצמו Claude. הוא בוחר ולמה Git פקודה – זו הזדמנות מצוינת לראות אילו פקודות.

**שלב 6 – בודקים.** מרעננים את עמוד המאגר ב-GitHub. ההודעה שכתב Claude מופיעה בהיסטוריה (לחיצה על **Commits**).

## עוד דוגמאות לבקשות

| מטרה | מה להקליד ב-Claude Code |
|---|---|
| להבין קוד | `הסבר לי שורה אחר שורה מה עושה @script.py` |
| לתקן שגיאה | `אני מקבל IndexError. כשאני מריץ את main.py מצא את הבעיה והסבר אותה לפני שאתה מתקן` |
| לכתוב בדיקות | `כתוב בדיקות לפונקציה calculate_grade` |
| לסכם שינויים | `סכם מה שיניתי מאז ה-commit האחרון` |
| ענף חדש | `צור ענף חדש בשם dark-mode ועבור אליו` |
| בקשת מיזוג | `create a pr for this feature` (יוצר Pull Request ב-GitHub; ייתכן שיידרש להתקין את GitHub CLI) |
| להתחיל מתכנון | `/plan להוסיף לאתר טופס צור קשר` |

בדיוק על איזה קובץ מדובר. אפשר גם לסמן שורות Claude-טיפ: הסימן `@` ואחריו שם קובץ אומר ל-Claude – בעורך רואה את הסימון.

## "גם ב"זמן מכונת-Claude Code

Claude Code שומר נקודות ביניים (Checkpoints) של העריכות שלו. מעבירים את העכבר מעל הודעה כדי להחזיר את הקבצים למצב שלפני השינוי. זה לא מחליף את Git: ובוחרים **Rewind code to here** רק commit ששלחתם ל-GitHub שמור בענן.

### כללים פדגוגיים לשימוש ב-AI בכתיבת קוד

- להסביר כל Claude-**קודם להבין, אחר כך לאשר.** אל תאשרו קוד שאינכם יכולים להסביר. בקשו מ שינוי.
- **יכול לטעות.** הריצו ובדקו כל דבר שהוא כותב **Claude**.
- **שקיפות.** אם בעבודה הוגשה עזרה של AI – ציינו זאת, לפי מדיניות בית הספר.
- **לא לשתף סודות.** אל תדביקו לשיחה סיסמאות או מידע אישי.

## 8. תרגיל מסכם, טעויות נפוצות ודף פקודות

### תרגיל מסכם (כ-45 דקות)

- [ ] התקינו Git והגדירו שם ואימייל ( `git config` ).
- [ ] פתחו חשבון GitHub והפעילו אימות דו-שלבי.
- [ ] צרו מאגר בשם `about-me` עם README.
- [ ] שכפלו אותו ל-VS Code.
- [ ] ערכו את `README.md` : (שם בדוי מספיק) כתבו שלוש שורות עליכם.
- [ ] עשו commit עם הודעה ברורה, ו-push.
- [ ] הוסיפו קובץ `hobbies.md` , עשו commit ו-push נוסף.
- [ ] ב-GitHub, לחצו על **Commits** וודאו שיש שתי נקודות שמירה.
- [ ] צרו ענף בשם `experiment` , שנו משהו, ובקשו מיזוג (Pull Request) לענף `main` .

### טעויות נפוצות ופתרונן

| הודעה / בעיה | מה זה אומר | פתרון |
|---|---|---|
| `git: command not found` | Git לא מותקן, או שהטרמינל נפתח לפני ההתקנה | התקינו Git, סגרו ופתחו מחדש את הטרמינל |
| `fatal: not a git repository` | אתם לא בתוך תיקיית המאגר | `cd` לתיקייה הנכונה, או `git init` |
| `Please tell me who you are` | לא הוגדרו שם ואימייל | הריצו את שתי פקודות `git config` מפרק 3 |
| `rejected ... fetch first` | יש שינויים שאין אצלכם ב-GitHub | `git pull` ואחר כך `git push` |

| הודעה / בעיה | מה זה אומר | פתרון |
|---|---|---|
| `Authentication failed` | ניסיון להתחבר עם סיסמה רגילה | התחברו דרך חלון הדפדפן, או דרך VS Code |
| `File ... exceeds 100 MB` | קובץ גדול מדי | הסירו אותו והוסיפו ל-`.gitignore` |
| הודעות commit כמו "asdf" | לא שגיאה, אבל הרגל רע | כתבו מה השתנה: "Add contact form" |

## דף פקודות (לתלות ליד המחשב)

| פקודה | מה היא עושה |
|---|---|
| `git --version` | בודק ש-Git מותקן |
| `git config --global user.name ".."` | מגדיר שם |
| `git init` | הופך תיקייה למאגר |
| `git clone <url>` | מוריד מאגר מ-GitHub |
| `git status` | מה השתנה? |
| `git add .` | מכין את כל השינויים לשמירה |
| `git commit -m ".."` | יוצר נקודת שמירה |
| `git push` | מעלה ל-GitHub |
| `git pull` | מוריד שינויים מ-GitHub |
| `git log --oneline` | היסטוריית נקודות השמירה |
| `git branch <name>` | יוצר ענף |
| `git switch <name>` | עובר לענף |
| `git merge <name>` | ממזג ענף לענף הנוכחי |
| `git diff` | מה בדיוק השתנה בכל שורה |

# מקורות

- GitHub Docs – About large files on GitHub
- GitHub Terms of Service
- GitHub Student Developer Pack – Terms
- Claude Code – Use Claude Code in VS Code
- Anthropic Consumer Terms of Service
- Git – הורדה רשמית